

Drone Security Analyst Agent

Submission Report — FlytBase AI Engineer Assignment

Jayesh Shete | jayeshnshete1204@gmail.com | June 2026

Repository	github.com/Jayesh1512/Flying-CCTV
Demo video	loom.com/share/edf1db4b1fe8478a9639fade8c22fe8e
Trial videos	Google Drive — drive.google.com/drive/u/0/folders/1Xj53ns2BOVryUWKgidTWby3QNLLKiPzH
VLM	Salesforce/blip-image-captioning-base (local, CPU/MPS)
Agent	LangChain 0.3 ReAct + GPT-4o-mini
Storage	SQLite (events) + ChromaDB (vector frames)
Alerts	5 deterministic rules (RULE-01 – RULE-05)

1. Feature Specification

The Drone Security Analyst Agent turns a fixed overhead camera into a continuously reasoning security observer. It replaces passive recording with active awareness: every frame with motion is described in plain English, deterministic rules fire before any LLM is consulted, and anomalies are pushed to an operator's phone within seconds.

Value proposition

- **Automated perimeter awareness** — person or vehicle detected, described, and logged every frame with motion.
- **Immediate, reliable alerts** — HIGH-severity events (person at night, loitering, perimeter breach) send an annotated photo to Telegram within seconds; no manual review required.
- **Queryable history** — operators ask questions in plain English after a session: *'How many men were detected?'*, *'Any vehicles near the garage?'*

Key requirements

#	Requirement	Implemented as
R1	Process video frame-by-frame and identify moving objects	MOG2 background subtraction → BLIP captioning on foreground crops
R2	Generate real-time alerts based on predefined security rules	5 deterministic rules (RULE-01 to RULE-05) fire before LLM; HIGH alerts → Telegram
R3	Index frames so they are searchable by content and time	ChromaDB (384-dim HNSW cosine) for semantic search; SQLite for structured queries

2. Design & Architecture

The pipeline is deliberately linear: each stage receives the previous stage's output type, with no shared mutable state. The LLM is the *last* stage so that deterministic rules cannot be overridden by a hallucination.

End-to-end flow

Stage	Component	Output type
Frame sampling	OpenCV VideoCapture (every 2nd raw frame)	np.ndarray
Background subtraction	MOG2 (5-frame warmup, VAR_THRESHOLD=120)	List[ObjectCrop]
Lighting detection	HSV V-channel mean + dark-pixel ratio	LightingAnalysis
VLM captioning	BLIP (blip-image-captioning-base, local)	FrameVLMResult
Object tracking	VehicleTracker (spatial proximity)	TrackUpdate
Alert rules	5 deterministic rules, deterministic order	List[Alert]

Stage	Component	Output type
ReAct agent	LangChain 0.3 + GPT-4o-mini + 4 tools	one-sentence summary
Storage	SQLite (events/alerts) + ChromaDB (frames)	persisted
Notification	Telegram Bot API, fire-and-forget thread	annotated JPEG

Key architectural decisions

MOG2 over YOLO for motion detection

MOG2 background subtraction runs entirely on CPU without a GPU, requires no labelled training data, and perfectly answers 'is anything moving?' — the only question needed to decide whether to invoke BLIP. YOLO would add 200–400 ms per frame and require a pretrained checkpoint that may not generalise to drone-mounted camera angles.

BLIP over GPT-4V / LLaVA / Moondream

BLIP runs locally (no per-frame API cost), produces consistently short declarative sentences ('a man is walking down the sidewalk with an umbrella') that keyword-extract cleanly, and handles the low-resolution 200–400 px crops that MOG2 produces. GPT-4V would be more accurate but costs ~\$0.01 per frame and adds 1–2 s latency. Moondream and LLaVA both exceeded 4 s / frame on CPU.

Deterministic rules before LLM

Alert decisions must be auditable and repeatable. The five rules evaluate a fixed set of signals (object_type, is_night, activity, entry_count, location) and fire before the ReAct agent runs. The agent receives already-fired alerts as context and can add narrative but cannot suppress a rule outcome.

Dual database

SQLite answers structured queries (severity filter, time range, exact alert lookup) in microseconds with zero infrastructure. ChromaDB answers semantic queries ('show me frames near the door') via 384-dimensional HNSW cosine similarity — something SQL LIKE cannot do meaningfully. The query engine routes each question automatically.

3. Implementation

Project structure

Module	Responsibility
vlm/background_subtractor.py	MOG2 wrapper; contour filtering by area
vlm/blip_analyzer.py	BLIP captioning; keyword extraction (object_type, color)
vlm/lighting_detector.py	HSV brightness → is_night flag
agent/alert_rules.py	RULE-01 to RULE-05 deterministic engine
agent/security_agent.py	LangChain ReAct agent (create_react_agent + AgentExecutor)
agent/tools.py	log_event, trigger_alert, query_history, query_track_positions

Module	Responsibility
agent/vehicle_tracker.py	Spatial proximity tracker; VehicleTracker + TrackUpdate
storage/event_store.py	SQLite wrapper (events + alerts tables)
storage/frame_index.py	ChromaDB wrapper (384-dim cosine, HNSW)
query/query_engine.py	NL query router (semantic vs structured)
notifications/telegram_notifier.py	Telegram Bot API; fire-and-forget daemon thread
pipeline/video_processor.py	Frame-by-frame orchestration; PipelineEvent stream
config.py	All constants; dotenv loading
main.py	CLI entry point
ui/app.py	Streamlit web interface

Custom components built during development

LightingDetector

Extracts the central 65% crop of the V-channel (HSV), computes mean brightness and dark-pixel ratio. `is_night = mean < 90 OR dark_pixel_ratio ≥ 0.55`. This avoids wall-clock time-of-day assumptions and works on footage recorded at any time.

VehicleTracker

Matches each new bounding-box center to the nearest active track within `max_match_distance = MAX_VELOCITY × frame_gap × MARGIN`. A miss creates a new track (`new_entry = True`); hits extend the existing trajectory. Used by RULE-03 to count repeat entries.

Union-find deduplication on count queries

When the query engine answers 'how many people?', overlapping track IDs from the same physical person are merged via union-find before counting — preventing double-counting from track restarts caused by MOG2 blob fragmentation.

SafeStdOutCallbackHandler patch

langchain-core 0.3 began passing `serialized=None` to `on_llm_start`. The default handler throws `TypeError`. A minimal subclass guards the `None` case, making LangChain logs safe to stdout.

Telegram notifier

Runs in a daemon thread so it never blocks the pipeline. Draws a red bounding box and caption over the triggering frame using OpenCV, encodes as JPEG in-memory, and posts via `sendPhoto`. Added at architecture stage (not retrofitted) because the threat model requires operators to receive alerts on their phone within seconds, not hours later when reviewing logs.

4. Alert Rules

All five rules are evaluated deterministically on every detected frame, in a fixed order, before the LangChain agent runs. The agent receives `pre_alerts` as input context.

Rule	Condition	Sev.	Rationale
RULE-01	person AND is_night	HIGH	Uninvited person at night is the single highest-risk scenario for a property
RULE-02	vehicle AND new_entry	MED	Any unrecognised vehicle entering should be flagged for operator review
RULE-03	vehicle entry_count > REPEAT_ENTRY_LIMIT	MED	Repeated entries in one session (default: >1) suggests casing behaviour
RULE-04	activity in {loitering, stationary}	HIGH	Stationary person at any time; fires on first sight and again after LOITER_THRESHOLD_SECONDS
RULE-05	location = perimeter AND is_night	HIGH	Any activity at perimeter in darkness — highest risk zone

Rules deliberately left out

- **Face recognition** — requires a known-faces database; false positives could flag authorised employees. Scope risk outweighed the value for a prototype.
- **Speed / trajectory rules** — meaningful only with calibrated camera-to-ground mapping. Not feasible without drone altitude and lens data.
- **Audio triggers** — video-only input; out of scope.

5. Results

All output below was generated by running the live pipeline on **outside_entry_720p.mp4** (C-MOR outdoor entrance clip, daytime, 84 raw frames).

5.1 BLIP frame descriptions

Frame	Detected object	BLIP caption	Alert fired
30	person	a man is seen in this surveillance image	RULE-04
32	person	a man is walking down the sidewalk with a camera	—

5.2 Annotated frames



Frame 30 — RULE-04 fired



Frame 32 — no alert

5.3 LangChain ReAct agent trace — Frame 22

Frame 022 → 1 object detected Objects : person Desc : a man is seen in a surveillance photo Activity : entering | Threat: high Track : track_002 (new object #2) Action: query_track_positions('main_gate') → Frame 21: center=(163.5, 279.0), track=track_001 → Frame 20: center=(150.5, 274.5), track=track_001 Thought: Current bbox center is far from track_001's last position – this is a new individual. Action: log_event('A new person was detected entering the main gate.') → Logged: evt_a4f2 Final Answer: A new person was detected entering the main gate.

5.4 Event log + ChromaDB semantic search

EVENT LOG (48 events) [HIGH] Frame 14 | a man is seen in this surveillance image [main_gate] [HIGH] Frame 23 | a man walking down a street with a black umbrella [main_gate] [HIGH] Frame 33 | a man is sitting on the floor in front of a door [main_gate] ...48 total events across 24 detected frames SEMANTIC SEARCH – Query: 'person at door' [score=0.61] Frame 33 | a man is sitting on the floor in front of a door [score=0.30] Frame 40 | a small window with a small window on the side NL QUERY – 'How many men were detected?' Frame 14 | a man is seen in this surveillance image (score=0.292) Frame 21 | a man is seen in this surveillance image (score=0.292) Frame 30 | a man is seen in this surveillance image (score=0.292) SESSION COMPLETE – 24 frames analyzed, 84 raw frames, 7.0 fps

6. AI Assistance Documentation

Two AI tools were used simultaneously throughout the project with deliberately separate roles. This was not a coincidence — it emerged from recognising that the two tools have fundamentally different context windows.

Tool	Role	Specific contributions
Codex	System-level design, cross-file debugging, documentation	Pipeline architecture; numpy/langchain conflict fix (two-step install); load_dotenv() ordering bug; SafeStdOutCallbackHandler patch; brightness-based lighting detection; all docs (PRD, ARCHITECTURE, BUILD_PLAN, README, CODE_DOCS)
Cursor AI	In-file implementation	Function bodies after interfaces were designed; SQLite query methods; LangChain @tool stubs; unit test boilerplate

Collaboration workflow

Codex → design interface + algorithm Cursor → implement function bodies Codex → wire modules, debug integration Cursor → in-file fixes Codex → documentation

The split mattered: Cursor occasionally bypassed config.py and wrote os.getenv() directly, or used a local constant instead of the shared one. Codex caught these on integration because it holds the full project in context.

PRD and architecture first

Before writing any code, Codex produced a PRD, a full architecture diagram, and a build plan. Telegram notifications were specified at this stage — not retrofitted — because the threat model required real-time operator alerting as a first-class requirement.

Dry-running edge cases before implementation

For each component, I described failure scenarios to Codex before writing code. This surfaced design constraints that would otherwise appear mid-debug:

Scenario	Constraint discovered
Same person re-enters the frame	Per-track deduplication needed, not per-frame counting
MOG2 fragments one object into 3 blobs	Minimum area filter (2000 px ²) + union-find dedup on count queries
Vehicle shadow larger than vehicle	detectShadows=True + threshold at 200 in MOG2
No warmup frames	5-frame silent learning period before any detection fires
LangChain calls log_event twice per frame	Explicit 'call exactly once' constraint added to system prompt

Custom Codex setup

- Installed **code-documentation skill** from skillcreatorai/Ai-Agent-Skills into .codex/skills/ — generates structured module-level docs on demand.
- Pre-approved permissions in .codex/settings.local.json: pip install *, yt-dlp *, WebFetch for arxiv / huggingface / moondream, and Skill(deep-research) — eliminated repeated prompts during the research phase.
- Cursor project rules: all constants from config.py, type annotations required, no trivial comments.

7. Cross-Domain Indexing

Two databases run in parallel — each chosen for what it does well:

SQLite — structured events

Every detected frame produces one row in the events table (frame_id, timestamp, location, severity, description, object_type, track_id). Alerts get their own table with a foreign key to events. This answers time-range queries and exact severity filters in microseconds.

ChromaDB — semantic frame index

BLIP captions are embedded using all-MiniLM-L6-v2 (384-dimensional) and stored in an HNSW cosine index. A query like 'person near the door' retrieves the nearest-neighbour frames without any keyword match — SQL LIKE cannot do this.

Query routing

The QueryEngine inspects the question and routes to the appropriate backend: structured keywords (count, alert, time, when) → SQLite; everything else → ChromaDB. Both results are merged and deduplicated before returning to the operator.

8. Testing

The test suite is split into fast unit tests (no model, no API) and optional integration tests that load BLIP and run the full pipeline.

Unit tests — 64 tests, ~2 seconds

Suite	What is tested
tests/unit/test_alert_rules.py	All 5 rules; boundary conditions; combinations
tests/unit/test_vehicle_tracker.py	Track matching, miss → new track, trajectory continuity
tests/unit/test_query_engine.py	SQLite routing, ChromaDB routing, dedup logic
tests/unit/test_event_store.py	Insert, retrieve, filter by severity, time range
tests/unit/test_lighting_detector.py	HSV brightness thresholds; dark-pixel ratio

Integration tests

ValidationCaptureRecorder runs the full pipeline against a known clip and records expected output as a golden fixture. On subsequent runs, the pipeline output is diffed against the fixture — any regression in BLIP caption, rule firing, or track assignment fails the test. Run with: `python3 -m pytest tests/integration -m integration -v`

AI-assisted test generation

Unit test boilerplate was generated by Cursor given the function signature and a short description of the expected behaviour. Codex reviewed boundary conditions (e.g., RULE-04 loitering window edge, union-find with single-node components) that Cursor missed.

9. Future Improvements

Improvement	Why	Effort
Replace BLIP with Moondream 2 (quantised)	7× more descriptive captions; fits 4 GB VRAM; 0.5 s/frame on GPU	Low
Session video summarisation	One-sentence summary after processing (e.g. 'No incidents in 2-hour patrol')	Low
Calibrated camera-to-ground mapping	Enables speed rules and precise GPS-tagged event locations	Medium
Face recognition with known-faces DB	Distinguishes authorised employees from strangers; eliminates RULE-01 false positives	Medium
Streaming ingestion (RTSP → Kafka)	Replace file-based VideoCapture with live drone feed for real deployment	High
Multi-camera session merging	Correlate events across multiple drone vantage points in one session	High
LangGraph stateful agent	Replace AgentExecutor with LangGraph for durable cross-session memory and branching	Medium

10. Assumptions

Assumption	Justification
Video files are pre-recorded MP4s, not live RTSP streams	Simplifies pipeline; live streaming is a deployment concern, not an AI-engineering concern for this prototype
No GPU required for VLM inference	BLIP runs on CPU (MPS on Apple Silicon); keeps the prototype accessible without specific hardware
A single camera angle per session	Multi-camera correlation was out of scope; the agent's memory window (k=10) is sufficient for one feed
Operator has Telegram and an OpenAI key	Both are optional — pipeline degrades gracefully (no Telegram → SQLite only; no OpenAI → rules still fire)
'Night' is defined by scene brightness, not wall-clock time	The drone may fly at dusk or in shadowed areas; HSV V-channel is more reliable than time-of-day